Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №9

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 6
Листинг: 7
lab9.c 7

Задание: протестируйте спин-блокировку используя фрагменты кода лекции 9.

Цель: знакомство с синхронизацией потоков.

Ход работы:

Спин-блокировка — низкоуровневый примитив синхронизации, применяемый в многопроцессорных системах для реализации взаимного исключения исполнения критических участков кода с использованием цикла активного ожидания. Применяется в случаях, когда ожидание захвата блокировки предполагается недолгим либо если контекст выполнения не позволяет переходить в заблокированное состояние.

```
char sh[6];
void* Thread( void* pParams );

int main( void ){
  pthread_t thread_id;
  pthread_create(&thread_id, NULL, &Thread, NULL);
  ……..
  while( 1 ) printf("%s\n", sh);
  ……..
}
```

```
void* Thread( void* pParams ){
  int counter = 0;
  while ( 1 ){
    if(counter%2){
      sh[0]='H';sh[1]='e';sh[2]='l';sh[3]='l';sh[4]='o';sh[5]='\0';
    }
    else{
      sh[0]='B';sh[1]='y';sh[2]='e';sh[3]='_';sh[4]='u';sh[5]='\0';
    }
    counter++;
  }
  return NULL;
}
```

Рис. 1 -Фрагменты кода к лабораторной 9.

Допишем программу (код представлен в листинге):

1. **Инициализация спин-блокировки:**
   - Мы добавили инициализацию спин-блокировки с помощью функции **pthread_spin_init** для переменной **lock** перед созданием потока.

2. **Использование функций спин-блокировки:**
    - В функции **Thread**, перед изменением значения **sh**, мы использовали **pthread_spin_lock(&lock)** для захвата спин-блокировки и **pthread_spin_unlock(&lock)** после изменения.
3. **Использование спин-блокировки в основном потоке:**
    - В основном цикле **main** мы также использовали спин-блокировку для защиты вывода **sh** на экран. По аналогии с потоком, мы захватываем и освобождаем спин-блокировку вокруг операции вывода.
4. **Уничтожение спин-блокировки:**
    - В конце программы мы добавили вызов **pthread_spin_destroy(&lock)** для корректного освобождения ресурсов, связанных со спин-блокировкой, после завершения работы.

Кроме того, мы добавили задержку в 0.5 секунды после изменения переменной **sh**. Это сделано для замедления процесса изменения значений и лучшей наглядности в выводе.

Аналогично в функции **main** добавили задержку в основном потоке вывода после каждого вывода значения переменной **sh**. на 0.1 секунды между итерациями цикла. Это сделано для более четкого наблюдения смены значений.

```
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab9$ gcc -o lab9 -Wall lab9.c
nothomepc@DESKTOP-M12GO9R:~/code/OS/lab9$ ./lab9

Bye_u
Bye_u
Bye_u
Bye_u
Hello
Hello
Hello
Hello
Hello
Bye_u
```

Рис. 2 – Компиляция и результат работы программы.

Вывод:

- Мы познакомились с синхронизацией потоков, узнали, что такое спин-блокировка.
- Доработав фрагменты кода лекции 9, протестировали спин-блокировку.

Листинг:

lab9.c

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

char sh[6];
pthread_spinlock_t lock;

void* Thread(void* pParams) {
    int counter = 0;
    while (1) {
        pthread_spin_lock(&lock);
        if (counter % 2) {
            sh[0] = 'H'; sh[1] = 'e'; sh[2] = 'l'; sh[3] = 'l'; sh[4] = 'o';
sh[5] = '\0';
        } else {
            sh[0] = 'B'; sh[1] = 'y'; sh[2] = 'e'; sh[3] = '_'; sh[4] = 'u';
sh[5] = '\0';
        }
        counter++;
        pthread_spin_unlock(&lock);
        usleep(500000);
    }
    return NULL;
}

int main() {
    pthread_t thread_id;
    pthread_spin_init(&lock, PTHREAD_PROCESS_PRIVATE);
    pthread_create(&thread_id, NULL, &Thread, NULL);

    while (1) {
        pthread_spin_lock(&lock);
        printf("%s\n", sh);
        pthread_spin_unlock(&lock);
        usleep(100000);
    }

    pthread_join(thread_id, NULL);
    pthread_spin_destroy(&lock);
    return 0;
}
```